

AXM Genesis: A Frozen Cryptographic Kernel for Verified Knowledge Reconstruction

Jonathan Sandhu
AXM.tools
bigbirdreturns@proton.me

Draft v0.6, June 2026

Abstract

We present AXM Genesis, a post-quantum signed knowledge format and reference verifier that removes stochastic language-model inference from the query-time critical path. A shard compiles source material into structured claims with byte-range provenance, Merkle-tree integrity, and ML-DSA-44 signatures. We define five conformance requirements, ship a 13-test verification suite, and show that deterministic query over signed shards achieves reproducibility, tamper detection, and offline operation by construction.

The central claim is architectural: improving retrieval cannot eliminate output entropy introduced after retrieval. In RAG, the runtime equation is a model call over a prompt. In AXM Genesis, the runtime equation is deterministic query over a verified shard. The inference call is not a primitive. It is an implementation choice that can be moved out of the critical path.

The gold shard, fm21-11-hemorrhage-v1, is the correctness oracle for the kernel. A conforming verifier accepts the frozen gold shard and rejects the invalid test vectors. Its statistics are manifest-declared and repo-verifiable; the count is not used as a benchmark.

Artifact: <https://github.com/BigBirdReturns/axm-genesis>. Commit: 6fa74b6d.

Artifact availability. The reference implementation, verifier, conformance suite, frozen gold shard, and invalid test vectors are available at <https://github.com/BigBirdReturns/axm-genesis>. This paper refers to commit 6fa74b6d. Full commit: 6fa74b6d8de87bf8e4f3cc00309e409704b806e4. The repository manifest and committed shard files are normative; product copy and website visuals are non-normative.

1 Introduction

Language models are increasingly used as interfaces to external knowledge. Because model parameters are incomplete, stale, opaque, and difficult to audit, deployed systems now use retrieval-augmented generation: retrieve relevant external text, place it in the model context, and ask the model to generate an answer. This architecture has become the default pattern for document question answering, internal knowledge assistants, compliance workflows, and enterprise search.

RAG is often described as a practical solution to hallucination. By grounding the model in retrieved documents, the system is expected to produce answers that are more accurate, up to date, and source-aware than a model answering from parameters alone. The canonical formulation combines parametric memory in a sequence-to-sequence model with non-parametric memory in a dense vector index.

These improvements are useful, but they preserve a core architectural commitment: **runtime generation**. In conventional RAG, the answer is not retrieved. It is generated. The retrieved

evidence constrains the generation, but the final answer remains the output of a probabilistic model. That distinction matters.

Evidence selection and answer reconstruction are separate stages. Improving selection can reduce evidence error, but it does not by itself eliminate variance introduced after selection. A system that requires repeatable answers must therefore control the runtime processor, not only the retriever.

For use cases such as compliance, legal audit, operational safety, autonomous systems, field manuals, procurement, medical protocols, regulated records, and engineering traceability, the central requirement is often not fluency. It is **repeatability**. The system must answer the same question the same way against the same evidence. It must identify exactly what source material supports the answer. It must detect tampering. It must operate offline when needed. It must preserve custody of the underlying knowledge.

1.1 Contributions

This paper makes four contributions:

1. A compact reconstruction-equation framing that separates selection, composition, and processing, and shows RAG as a runtime-generation special case (sections 3 and 4).
2. An entropy-and-variance argument showing that improving context selection cannot guarantee deterministic reconstruction when the runtime processor has non-zero output entropy (section 5).
3. AXM Genesis: an implemented VRA with a frozen cryptographic kernel, post-quantum signatures, five conformance requirements, a 13-test verification suite, and a gold shard correctness oracle (section 6).
4. A proof-by-construction evaluation showing categorical separation between AXM and RAG on reproducibility, tamper detection, offline capability, and marginal query cost (section 7).

2 Background and Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation was introduced as a method for combining parametric language-model memory with non-parametric external memory [1]. In the canonical formulation, a retriever selects passages from a large corpus and a generator produces an answer conditioned on both the query and retrieved passages. The retriever is commonly implemented through dense vector similarity over embedded text passages. The generator is a sequence-to-sequence or causal language model.

The standard RAG pipeline has four stages:

1. **Ingestion:** source documents are split into chunks.
2. **Embedding:** chunks are converted into vector representations.
3. **Retrieval:** a query embedding is matched against the vector index.
4. **Generation:** retrieved chunks are placed into a prompt and passed to a language model.

Subsequent work improves RAG by targeting the retrieval side: hybrid search, rerankers, knowledge-graph RAG, multi-hop retrieval, and agentic orchestration [2, 3]. These systems improve the context selected for a query. However, most preserve runtime generation by an LLM. The answer is still produced by a stochastic processor after retrieval.

2.2 Provenance and Signed Data Structures

Separate from the RAG literature, computer systems have long used cryptographic hashes, Merkle trees, signatures, content addressing, append-only logs, reproducible builds, and signed artifacts to establish integrity [5, 6]. These techniques make data tampering detectable. Verification is performed by recomputing hashes and checking signatures.

AXM applies this systems tradition to knowledge artifacts. A shard is not merely a retrieved document collection. It is a signed knowledge artifact containing extracted claims, source spans, manifest metadata, and cryptographic commitments that bind the output of compilation to the source material.

3 Reconstruction Equations

3.1 Runtime Generation in One Equation

A conventional RAG system can be summarized as:

$$\text{Output}_{\text{RAG}}(q) = L_{\theta}(\text{Prompt}(q, C(q)))$$

Here:

- q is the user’s query.
- $C(q)$ is the context retrieved for that query.
- $\text{Prompt}(q, C(q))$ is the prompt assembled from the query and retrieved context.
- L_{θ} is the runtime language model that generates the answer.

This equation captures the architectural commitment of RAG. Even when retrieval succeeds, the final answer is generated by a runtime model call. Better retrieval changes $C(q)$. It does not remove L_{θ} from the answer path.

3.2 AXM in One Equation

AXM Genesis changes the runtime equation:

$$\text{Output}_{\text{AXM}}(q) = \text{Query}(q, S_{\text{verified}})$$

Here:

- S_{verified} is a signed shard that has passed manifest validation, Merkle verification, signature verification, schema checks, and provenance checks.
- Query is a deterministic query procedure over the verified shard.
- The runtime processor is no longer a language model. It is a verifier-backed query engine operating over a cryptographically committed artifact.

A language model may still be used during compilation, extraction, summarization, or query translation. The difference is authority. In AXM, query-time authority comes from a verified shard and deterministic execution, not from runtime generation.

3.3 The General Pipeline

The two equations are instances of the same reconstruction pipeline:

Store $\rightarrow$ Select $\rightarrow$ Compose $\rightarrow$ Process $\rightarrow$ Output

- Store contains documents, claims, tables, graphs, sensor logs, code, policies, images, transcripts, field manuals, contracts, or other source material.
- Select chooses the material relevant to a query.
- Compose converts the selected material into the input consumed by the processor.
- Process produces the answer.
- Output is the answer returned to the user or downstream system.

This pipeline separates two questions that RAG often collapses:

1. What evidence was selected?
2. What process produced the answer from that evidence?

A **Verifiable Reconstruction Architecture** (VRA) is a reconstruction pipeline whose output can be checked against its source material and whose relevant source material can be independently verified. A VRA should provide:

1. **Source binding:** outputs are traceable to specific source units.
2. **Integrity:** source units and derived structures are protected against undetected tampering.
3. **Determinism option:** the query-time processor may be deterministic.
4. **Local verification:** verification does not require trust in a remote model provider.
5. **Reproducibility:** given the same verified source, query, and deterministic processor, the output is identical.

These properties separate verified reconstruction from ordinary retrieval-augmented generation. A RAG answer may cite a document, but the generated text is not itself a signed reconstruction from source. It is a model output conditioned on source.

4 RAG as a Degenerate Runtime-Generation Case

4.1 RAG as an Instance of the Pipeline

Standard RAG sets the pipeline stages as follows:

- Store: text chunks, usually embedded into a vector index.
- Select: semantic or hybrid retrieval, producing $C(q)$.
- Compose: prompt construction, written as $\text{Prompt}(q, C(q))$.
- Process: runtime generation by L_θ .
- Output: generated text.

That yields the runtime-generation equation:

$$\text{Output}_{\text{RAG}}(q) = L_\theta(\text{Prompt}(q, C(q)))$$

RAG is therefore not outside reconstruction architectures. It is one setting of the pipeline: retrieval for selection, prompt stuffing for composition, and a language model for runtime processing.

4.2 Degeneracy

A special case is not automatically degenerate. It becomes degenerate when it unnecessarily commits to constraints that are harmful for the target domain.

RAG becomes degenerate for high-assurance reconstruction because it commits to:

1. **Chunk-level knowledge representation:** source material is stored as text chunks rather than normalized claims or typed relations.
2. **Semantic similarity as primary selection:** retrieval may be approximate, relevance-ranked, and unstable under paraphrase or embedding drift.
3. **Prompt stuffing as composition:** context is converted into unstructured natural-language input.
4. **Stochastic generation at runtime:** final answers are produced by a probabilistic model.
5. **Weak output provenance:** generated answers may cite documents but are not mechanically derived from signed source spans.
6. **Recurring inference cost:** every answer requires a model invocation when generation remains on the runtime path.
7. **Provider dependency:** deployed systems depend on remote inference APIs, rate limits, model versions, and hidden updates.

Proposition 4.1 (RAG Degeneracy). Standard RAG is a runtime-generation instance of a reconstruction architecture, but it is degenerate for deterministic reconstruction when L_θ remains a stochastic language model and composition remains natural-language prompt construction.

Proof Sketch. RAG satisfies the reconstruction pipeline because it selects context, composes a prompt, and processes that prompt into output. However, runtime stochastic generation means the output is not a deterministic function of the selected evidence unless generation is constrained to deterministic execution and all model/provider nondeterminism is removed. Natural-language prompt composition also weakens the mechanical relation between source span and answer. Therefore, for deterministic reconstruction, RAG places the least stable processor at the most critical stage: query time. $\square$

5 Runtime Entropy and Reconstruction Variance

5.1 Problem Statement

Improving retrieval does not guarantee reproducible reconstruction when the final step is stochastic generation. The retrieved evidence may be correct and complete, while the generated answer still varies across runs, model versions, providers, decoding settings, or hidden runtime changes.

GraphRAG, hybrid retrieval, rerankers, multi-hop retrieval, and better embeddings improve $C(q)$. They do not remove variance introduced by L_θ when L_θ remains a stochastic runtime processor.

Informally, this is the sand problem: changing the container does not control the grain-level fall pattern. In this paper, the analogy is only shorthand. The formal issue is output entropy in the runtime processor.

5.2 Formal Statement

For a fixed query q and fixed retrieved context $C(q)$, define the runtime prompt as:

$$x = \text{Prompt}(q, C(q))$$

If the runtime model is stochastic, then the output is sampled from a distribution:

$$L_\theta(x) \sim D_x$$

A deterministic reconstruction requires a single repeated output for the same query and same verified evidence. If the output distribution has non-zero entropy,

$$H(D_x) > 0,$$

then there are at least two distinct outputs y_1 and y_2 with non-zero probability. Therefore, the system cannot guarantee reproducible reconstruction unless the entropy of the runtime processor is eliminated or the output is post-processed into a deterministic form.

5.3 Theorem

Theorem 5.1 (Runtime Entropy Bound). Improving context selection cannot guarantee deterministic reconstruction if the runtime processor remains stochastic with non-zero output entropy.

Proof. Hold the query q and selected context $C(q)$ fixed. The composed prompt $\text{Prompt}(q, C(q))$ is then fixed as well. The only remaining source of output variation is the runtime processor. If that processor is stochastic with non-zero output entropy, repeated evaluations may yield different outputs. Improving context selection cannot remove variance introduced after selection. Therefore, retrieval improvements alone cannot guarantee deterministic reconstruction. $\square$

Corollary 5.1. Systems requiring deterministic, auditable, or legally reproducible answers should not place stochastic language-model generation on the critical runtime path.

This does not prohibit language models. It relocates them. A language model may be used during compilation, extraction, summarization, candidate-claim generation, or query translation, where its output can be checked, signed, rejected, or corrected. It should not be the final authority at query time in high-assurance systems.

6 AXM Genesis: Proof by Construction

6.1 Overview

AXM Genesis is a Verifiable Reconstruction Architecture that implements the reconstruction equation without runtime language-model inference. AXM’s core thesis is:

LLM at compile time. SQL at query time.

Source material is compiled into signed shards. Querying a shard does not require a model. It requires a deterministic processor and local verification.

6.2 The Shard Format

A shard is a sealed knowledge artifact. Figure 1 shows its directory layout. Every claim traces to exact bytes in the source document. If any covered byte changes, the Merkle root changes, signature verification fails, and the shard is rejected.

```

shard/
  manifest.json      # Metadata, Merkle root, crypto suite
  sig/
    manifest.sig     # ML-DSA-44 (FIPS 204) or Ed25519
    publisher.pub    # Public key of signing publisher
  content/
    source.txt       # Primary document (byte-addressable)
    [domain files]   # e.g., cam_latents.bin for embodied
  graph/
    entities.parquet # Typed entities with deterministic IDs
    claims.parquet   # Structured claims with evidence links
    provenance.parquet # Lineage and compilation metadata
  evidence/
    spans.parquet    # Byte-range source spans
  ext/               # Domain extensions

```

Figure 1: Shard directory layout.

6.3 Cryptographic Kernel

6.3.1 Hash Functions

AXM uses BLAKE3 for Merkle-tree construction and SHA-256 for content hashing. BLAKE3’s internal binary tree structure provides efficient parallel hashing and natural Merkle-proof capability [7].

6.3.2 Signature Suites

Table 1: AXM Cryptographic Suites

Suite	Algorithm	Key Size	Sig Size	Default
Ed25519 (legacy)	Ed25519	32 B	64 B	Pre-v1.1.0
axm-blake3-mldsa44	ML-DSA-44 (FIPS 204)	1312 B	2420 B	v1.1.0+

For ML-DSA-44 compilation, the private key must be either:

- `sk||pk` concatenated blob: $2528 + 1312 = 3840$ bytes, or
- `sk` only: 2528 bytes, with matching `sig/publisher.pub` (1312 bytes) pre-placed.

Any other key length is rejected with `ValueError`.

6.3.3 Merkle Construction

The Merkle tree is constructed over a **covered file set** F defined as every file under the shard root except:

- `manifest.json` itself (it contains the Merkle root);
- files in `sig/` (the signature and public key are verified against the manifest, not included in it);
- dotfiles and hidden files (names beginning with `.`).

The covered set is ordered by **lexicographic path** (UTF-8 NFC, forward slashes as separators, case-sensitive). This ordering is deterministic across all implementations.

Construction proceeds as follows:

1. Each file $f \in F$ is hashed with BLAKE3, producing a 32-byte leaf hash $h_f = \text{BLAKE3}(f)$.
2. Leaf hashes are paired in order: $(h_1, h_2), (h_3, h_4), \dots$. If the count is odd, the last hash is paired with itself.
3. Each parent node concatenates its two children and hashes with BLAKE3, setting the PARENT flag in the BLAKE3 context.
4. The root node sets the ROOT flag.
5. The Merkle root is stored in `manifest.json` under `integrity.merkle_root`.

The manifest itself is canonicalized: UTF-8 NFC, sorted JSON keys, no whitespace, no trailing newline. This ensures deterministic reconstruction of the Merkle root across all implementations.

6.3.4 Spec-Change Rule

The AXM Genesis specification is versioned as `major.minor.patch`:

- **Patch** changes (e.g., `1.0.0` $\rightarrow$ `1.0.1`): clarifications, typo fixes, non-normative additions. No RFC required.
- **Minor** changes (e.g., `1.0.0` $\rightarrow$ `1.1.0`): new optional fields, backward-compatible schema extensions, new error codes. Requires frozen-spec RFC with 14-day comment period.
- **Major** changes (e.g., `1.0.0` $\rightarrow$ `2.0.0`): breaking changes to the shard format, Merkle construction, or cryptographic suite. Requires frozen-spec RFC with 30-day comment period and a **new gold shard** under a new namespace.

A gold shard is frozen within its major version line. `fm21-11-hemorrhage-v1` will never be recompiled within `v1.x`. A `v2.0` gold shard would use a different namespace (e.g., `fm21-11-hemorrhage-v2`) and a different Merkle root.

6.4 Compilation

Compilation transforms source material into a signed shard:

$$S_{\text{verified}} = \text{Compile}(\text{Source})$$

Here, `Source` may contain one document or many source files. The compiler produces a shard with structured claims, source spans, manifest metadata, Merkle commitments, and a publisher signature.

The compilation process:

1. Parse source files.
2. Normalize text (UTF-8 NFC).
3. Extract entities (may use LLM).
4. Extract claims (may use LLM).
5. Link claims to byte ranges.
6. Validate claim/source alignment.
7. Build Parquet tables with deterministic row ordering.
8. Compute file hashes (BLAKE3).

9. Construct Merkle root per section 6.3.3.
10. Sign manifest with publisher key.

A language model may be used in steps 3 or 4, but it is not trusted blindly. It is an extractor, not the runtime authority.

6.5 Runtime Querying

At runtime, the query process has three deterministic stages: verify the shard, translate the question into a query plan, and execute that query locally. In compact form:

$$\text{Output}_{\text{AXM}}(q) = \text{Query}(q, S_{\text{verified}})$$

Here:

- S_{verified} is the shard after manifest, Merkle, signature, schema, and provenance checks pass.
- Query is deterministic execution over the verified shard, typically SQL over Parquet tables.
- If a language model translates a natural-language question into SQL, the generated SQL is an intermediate artifact. The executed query remains deterministic.

The runtime authority path is therefore local and inspectable: verified shard bytes, deterministic query plan, deterministic query execution.

6.6 Verification

Before a query result is accepted, the shard is verified:

Algorithm 1. AXM Shard Verification

Input: Shard directory D, trusted public key pk

Output: ACCEPT or REJECT with error code

1. Parse manifest.json from D.
2. Verify manifest schema (REQ 1).
3. Enumerate covered file set F per Section 6.3.3.
4. Recompute BLAKE3 hash of every file f in F.
5. Reconstruct Merkle root from file hashes per Section 6.3.3.
6. If computed root != manifest root:
 - return REJECT with E_MERKLE_MISMATCH.
7. Load sig/manifest.sig and sig/publisher.pub.
8. If signature invalid:
 - return REJECT with E_SIG_INVALID.
9. Verify all Parquet schemas (REQ 2-3).
10. Verify entity and claim ID consistency (REQ 3).
11. Verify source span byte ranges (REQ 2).
12. If embodied spoke:
 - verify cam_latents.bin continuity (REQ 5).
13. Return ACCEPT.

Figure 2: AXM shard verification.

This makes integrity a local property. The user does not need to trust the model, API provider, cloud index, or remote service.

6.7 Conformance Requirements

REQ 5 applies to spokes maintaining binary hot streams (e.g., embodied robotics). Shards without cam_latents.bin pass silently.

Table 2: AXM Compatibility Requirements

Req	Description	Error Codes
REQ 1	Manifest integrity	E_SIG_INVALID,
REQ 2	Content identity	E_MERKLE_MISMATCH
REQ 3	Lineage events	E_MERKLE_MISMATCH,
REQ 4	Proof bundle	E_REF_SOURCE
REQ 5	Non-selective recording	E_REF_ORPHAN, E_SCHEMA_NULL
		E_SIG_INVALID, E_SIG_MISSING
		E_BUFFER_DISCONTINUITY

6.8 Conformance Test Matrix

The 13 conformance tests map directly to requirements:

Table 3: Conformance Test-to-Requirement Mapping

Test	Name	Target	Req
T1	Manifest schema violation	Invalid JSON, missing fields	REQ 1
T2	Merkle root mutation	Flip one bit in manifest root	REQ 1, REQ 2
T3	Content byte mutation	Modify one byte in <code>source.txt</code>	REQ 2
T4	Parquet byte mutation	Modify one byte in <code>claims.parquet</code>	REQ 2
T5	Orphaned claim reference	Claim points to non-existent entity	REQ 3
T6	Null required column	Null in non-nullable Parquet field	REQ 3
T7	Wrong signing key	Sign with untrusted key pair	REQ 1, REQ 4
T8	Signature bit-flip	Modify one bit in <code>manifest.sig</code>	REQ 4
T9	Missing manifest	Delete <code>manifest.json</code>	REQ 4
T10	Buffer discontinuity	Gap in <code>cam_latents.bin</code> frames	REQ 5
T11	Valid continuous stream	Correct AXLF/AXLR sequence	REQ 5
T12	Deterministic verification	Same shard, same result	REQ 1–4
T13	Layout violation	Extra file in required directory	REQ 1

A verifier is conformant when it passes T1–T13: accepts the gold shard and rejects all invalid vectors.

6.9 The Gold Shard

`shards/gold/fm21-11-hemorrhage-v1/` is the reference shard extracted from FM 21-11, the US Army first aid field manual. It defines correctness:

- Any verifier that **accepts** this shard and **rejects** the invalid test vectors in `tests/vectors/shards/invalid/` is conformant.
- The gold shard is frozen within the v1.x specification line. It will never be recompiled.
- The manifest is the normative source of truth for shard statistics. Product copy and website visuals are non-normative.

The committed manifest declares the following statistics and integrity anchor:

The gold shard is the immutable correctness oracle for the AXM ecosystem. The six-claim/eight-entity count is not a benchmark and is not presented as evidence of dataset scale. It is the exact scope of the frozen conformance artifact currently committed in the repository.

Table 4: Gold Shard Manifest Facts for fm21-11-hemorrhage-v1. The count is a conformance scope, not a scale claim.

Field	Manifest-declared value
Shard path	shards/gold/fm21-11-hemorrhage-v1/
Specification version	1.0.0
Claims	6
Entities	8
Hash algorithm	blake3
Merkle root	e30deb23...dcee21
Source file	content/source.txt
License	CC0-1.0

Table 5: AXM Satisfaction of VRA Properties

VRA Property	AXM Mechanism
Source binding	Claims link to source spans
Integrity	Hashes, Merkle root, signatures
Determinism	SQL or fixed query engine at runtime
Local verification	Offline signature verification
Reproducibility	Same shard + same query = same answer
Auditability	Claim and span tables preserve evidence
Cost shift	Inference cost moves to compilation

6.10 AXM as VRA

7 Evaluation by Construction

The evaluation in this paper is not a latency benchmark and does not rely on sampled quality scores. It is a proof-by-construction comparison between two architectural commitments. RAG places a stochastic language model on the query-time critical path. AXM Genesis places a deterministic verifier and query engine on that path. The resulting separation follows from the construction of the systems.

7.1 Objects Compared

We compare two classes of systems:

1. **Runtime-generation systems**, including standard RAG, GraphRAG, hybrid RAG, and agentic RAG, in which retrieved evidence is ultimately passed to a language model for answer generation at query time.
2. **Signed reconstruction systems**, represented by AXM Genesis, in which source material is compiled into a signed shard and query-time processing is deterministic over that shard.

The purpose of the comparison is not to show that AXM produces more fluent prose. It does not. The purpose is to show that AXM provides categorical guarantees that runtime-generation systems do not provide while a stochastic model remains in the answer path.

7.2 Proof-by-Construction Matrix

The table is not a benchmark table. It is a construction table. The RAG column follows from the fact that the answer is generated at runtime by a model. The AXM column follows from the

fact that the shard is signed, the verifier is deterministic, and query-time execution need not call a language model.

Table 6: Architectural Separation Between RAG Variants and AXM Genesis

Property	RAG (any runtime-generation variant)	AXM Genesis
Reproducibility	$\leq 1 - \epsilon$ when stochastic generation, provider drift, sampling, or non-deterministic inference remains in the answer path	1.0 for deterministic query over an unchanged signed shard
Tamper evidence	Not inherent; text chunks, embeddings, and retrieved contexts are not cryptographically bound by the RAG architecture itself	Perfect for covered artifact bytes: Merkle mismatch or signature failure rejects the shard
Offline query	Requires model weights or an inference endpoint; remote RAG requires network access and provider availability	Yes: Parquet tables plus local verifier/query engine
Per-query cost	$O(\text{tokens})$ plus model invocation whenever generation is used at runtime	Local file I/O and deterministic query; inference cost is moved to compilation
Provenance granularity	Usually chunk-level or citation-level; generated statements may not be mechanically bound to exact source spans	Byte-range exact: claims are linked to source offsets in the shard
Post-quantum security	Not a property of RAG	ML-DSA-44 suite available through axm-blake3-mldsa44

7.3 Formal Metrics

For completeness, the same separation can be expressed as metrics.

7.3.1 Reproducibility

Run the same query Q against the same artifact N times and collect outputs $O_1, \dots, O_N$. Define:

$$\rho(Q) = 1 \quad \text{iff} \quad O_1 = O_2 = \dots = O_N; \quad \rho(Q) = 0 \quad \text{otherwise.} \quad (1)$$

AXM Genesis achieves $\rho(Q) = 1$ for deterministic query over an unchanged verified shard. Runtime-generation systems achieve $\rho(Q) = 1$ only if the generation path is made fully deterministic and model/provider drift is excluded. That condition is not supplied by RAG itself.

7.3.2 Tamper Detection

Let A be a shard artifact and A' be the same artifact with one covered byte modified. Define:

$$\tau(A') = 1 \quad \text{iff} \quad \text{Verify}(A') = \text{REJECT}; \quad \tau(A') = 0 \quad \text{otherwise.} \quad (2)$$

For AXM Genesis, any covered byte modification changes the Merkle root or invalidates the signature. RAG has no equivalent architectural guarantee unless an external signing layer is added. At that point, the signing layer, not RAG, supplies the guarantee.

7.3.3 Marginal Query Cost

For runtime-generation RAG:

$$C_{\text{query}}^{\text{RAG}} = C_{\text{retrieval}} + C_{\text{tokens}} + C_{\text{model-runtime}}. \quad (3)$$

For AXM Genesis:

$$C_{\text{query}}^{\text{AXM}} = C_{\text{verify/cache}} + C_{\text{local-query}}. \quad (4)$$

After verification is cached, the marginal query path is local deterministic query over the shard. Exact latency depends on implementation and hardware; the architectural separation is that runtime inference is absent.

7.4 Conformance Evidence

The submitted implementation includes a conformance suite whose role is not to estimate quality but to define acceptance. A verifier is conformant when it accepts the gold shard and rejects invalid vectors. The 13 conformance tests (table 3) exercise manifest mutation, invalid JSON, content-byte mutation, Parquet-byte mutation, orphaned claims, null required columns, wrong signing keys, signature mutation, missing manifest, buffer discontinuity, valid continuous streams, and deterministic verification.

7.5 What This Evaluation Does Not Claim

This evaluation does not claim that AXM is better than RAG for open-ended synthesis, summarization, or creative answer generation. It claims categorical separation on properties that follow from the kernel: reproducibility, tamper evidence, offline verification, and absence of query-time inference. Those properties are the relevant ones for verified knowledge reconstruction.

8 Discussion

8.1 When RAG Is Appropriate

RAG remains appropriate when answers are exploratory, fluency matters more than exactness, the source corpus changes constantly, users expect synthesis rather than reconstruction, citations are useful but not legally binding, and runtime creativity is desired.

8.2 When VRA Is Appropriate

VRA is appropriate when answers must be reproducible, source custody matters, offline operation is required, audit trails must survive review, tamper detection is necessary, every claim must cite a source, runtime inference cost is unacceptable, and the system cannot depend on a model provider.

8.3 Compile-Time Use of Language Models

This paper does not reject language models. It rejects making them the final runtime authority in high-assurance reconstruction. Language models are useful at compile time: extracting candidate claims, summarizing source material, identifying entities, proposing relations, generating query plans, normalizing language, detecting contradictions. But compile-time output can be reviewed, corrected, rejected, signed, and frozen. Runtime generation cannot provide the same guarantees unless constrained into deterministic behavior and source-bound outputs.

8.4 Incentives

There is an economic reason that runtime-generation architectures dominate: they preserve the inference call. Every query can be billed, rate-limited, logged, updated, or mediated by a provider. VRA shifts cost to compilation. After compilation, queries are local. This is economically unattractive to model providers but valuable to users who need to own and verify their knowledge.

9 Limitations

Extraction quality. AXM depends on the quality of compiled claims. If extraction is poor, the shard will faithfully preserve poor structure. Mitigation: claim validation, human review, schema checks, redundant extraction, source-span verification.

Dynamic knowledge. VRA is strongest when source material is stable enough to compile. Rapidly changing corpora require incremental compilation and versioning.

Open-ended reasoning. AXM is not optimized for open-ended prose synthesis. A downstream language model may be used to explain results, but that explanation should be labeled as presentation, not the authoritative answer.

Query translation. Natural-language questions must be translated into deterministic queries. If an LLM generates SQL, the SQL should be displayed, validated, and executed deterministically.

Cryptographic scope. Signatures prove integrity and publisher identity. They do not prove that the original source is true. AXM provides provenance and tamper evidence, not omniscience.

10 Conclusion

RAG is useful, but it is not the general form of source-grounded knowledge systems. It is one runtime-generation implementation of a broader reconstruction architecture.

We defined Verifiable Reconstruction Architectures as systems that select context from a knowledge store, compose that context into an input, and process it into an output. The processing function need not be a language model. The context function need not be vector retrieval. The composition function need not be prompt stuffing.

Standard RAG becomes degenerate in domains requiring determinism, auditability, provenance, offline operation, and tamper detection because it leaves stochastic language-model inference on the critical runtime path. Better retrieval improves evidence selection, but it does not remove output variance introduced after selection.

AXM Genesis demonstrates the alternative. Compile knowledge once. Bind claims to source spans. Sign the shard with post-quantum cryptography. Query deterministically. Verify locally. Use language models where they are useful, at compile time, but remove them from the runtime authority path.

The practical implication is simple: not every knowledge system should answer by generating text. Some should answer by reconstructing verified knowledge.

The inference call is not a primitive. It is a choice.

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.

- [2] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented Language Model Pre-Training. *ICML*, 2020.
- [3] X. Zhu et al. Knowledge Graph-Guided Retrieval Augmented Generation. *NAACL*, 2025.
- [4] Matthijs Douze et al. The Faiss Library. *arXiv:2401.08281*, 2024.
- [5] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. *CRYPTO*, 1987.
- [6] W3C PROV Working Group. PROV-DM: The PROV Data Model. W3C Recommendation, 2013.
- [7] Jean-Philippe Aumasson et al. The BLAKE3 Hashing Framework. Internet-Draft, IETF, 2024.
- [8] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard. National Institute of Standards and Technology, 2024.
- [9] DuckDB Foundation. DuckDB: An In-Process Analytical Database. <https://duckdb.org>.
- [10] BLAKE3 Team. BLAKE3: One Fast Cryptographic Hash. <https://github.com/BLAKE3-team/BLAKE3>.